

APPEX LEAGUE

PLAY

INTRODUCTION

Apex League adalah game kompetitif berbasis fisika dimana pemain mengendalikan kendaraan bertenaga roket dalam arena dua dimensi yang sepenuhnya datar (flat). Fokus utama permainan adalah penguasaan momentum dan arah vektor untuk menggiring bola ke gawang lawan.

Game ini dirancang sebagai bukti kompetensi rekayasa perangkat lunak, dengan mengedepankan stabilitas mesin fisika. Tidak ada elemen ketinggian (sumbu-Z) dalam game ini, sehingga seluruh tantangan berpusat pada akurasi benturan dan timing manuver pada bidang datar.

CORE GAMEPLAY MECHANICS

Dash

Boost

Drift

Demolition



TECH STACK & ARCHITECTURE

Frontend (Client): Java menggunakan framework LibGDX + Box2D Physics Engine.

Backend (API Server): Spring Boot (Java) sebagai RESTful API handler.

Primary Database (Storage): PostgreSQL untuk integritas data persisten (User Profile, Car Stats, Match History).

In-Memory Cache (Speed): Redis (Sorted Sets) untuk komputasi Global Leaderboard secara instan.

DUAL-DATABASE STRATEGY FOR LEADERBOARD

Masalah: Mengurutkan ribuan data pemain di SQL konvensional sangat membebani server saat diakses terus-menerus.

Solusi Apex League: PostgreSQL difokuskan untuk mengamankan data riwayat secara ACID compliance.

- Saat pertandingan selesai, Backend secara otomatis mengirim nilai ke Redis ZSET (Sorted Set).
- Hasilnya: Data untuk tab MMR, WINS, GOALS, SAVES, dan DEMOS dapat di-fetch dalam hitungan milidetik.

DESIGN PATTERN IMPLEMENTATION

Component Pattern (ECS)

Memecah entitas game menjadi data (PhysicsComponent, InputComponent) dan logika (MovementSystem). Membuat pengembangan fitur jauh lebih fleksibel.

Observer Pattern

Diterapkan pada GameContactListener. Sistem mendeteksi tabrakan fisik (Goal, Demolition, Boost) secara otomatis tanpa mengikat komponen lain (decoupled).

Object Pool Pattern (Performance Booster)

Menggunakan BoostPadPool untuk mendaur ulang objek boost di arena. Terbukti mencegah lag akibat beban Garbage Collector.

Command Pattern

Mengubah input mentah pemain (WASD/IJKL) menjadi objek perintah abstrak (MoveCommand, DriftCommand), memastikan kontrol multiplayer sangat responsif.

Strategy & State Pattern

MovementSystem menukar handling mobil secara dinamis (Normal vs Drift), sementara GameManager mengatur alur pertandingan (Kickoff, Resetting, Game Over) dengan rapi.

Factory Pattern

Pabrik terpusat (CarFactory, ArenaFactory) yang menyederhanakan perakitan objek Box2D yang rumit menjadi satu baris kode.

DESIGN PATTERN IMPLEMENTATION

Strategy Pattern (Core Leaderboard)

Menghilangkan if-else raksasa. Sistem secara dinamis memilih strategi (misal: MmrLeaderboardStrategy atau GoalsLeaderboardStrategy) untuk menarik ranking langsung dari Redis ZSET.
(MmrLeaderboardStrategy.java, LeaderboardServiceImpl.java)

Observer Pattern (Event-Driven)

Bertugas menjaga integritas data. Redis hanya akan di-update secara atomik setelah riwayat pertandingan 100% aman tersimpan (Commit) di PostgreSQL. (RedisLeaderboardObserver.java)

Facade Pattern

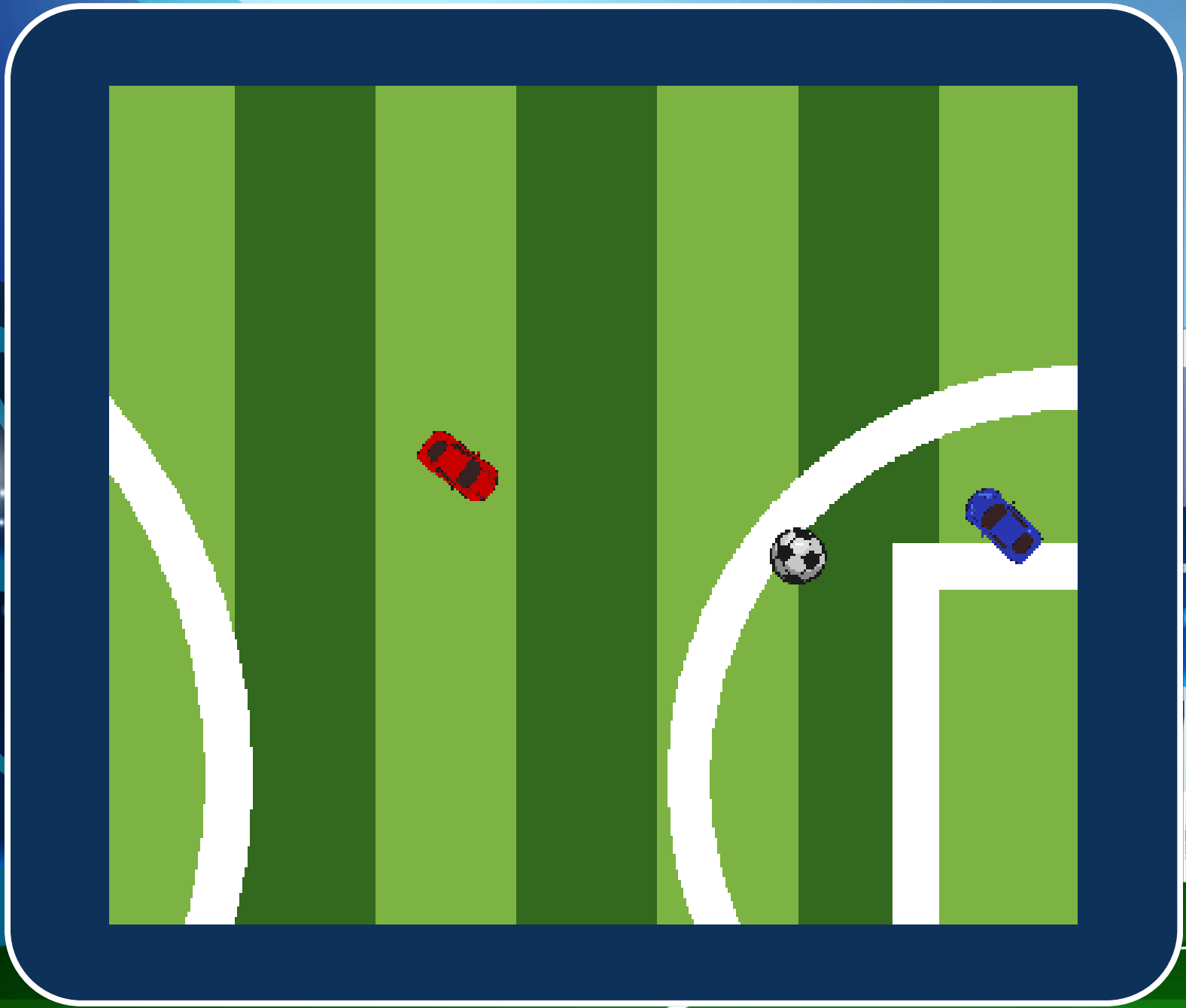
Controller (seperti LeaderboardController.java, MatchHistoryController.java) bertindak sebagai gerbang depan yang rapi. Frontend tidak perlu tahu kerumitan komunikasi antara Postgres dan Redis di belakang layar.

DAO / Repository Pattern

Abstraksi akses data menggunakan Spring Data JPA. Logika bisnis terbebas dari penulisan query SQL mentah.
(MatchHistoryRepository.java)

Singleton Pattern

Bawaan dari Spring Boot (@Service, @Component). Menjamin operasi database dan Redis berjalan di instansi tunggal yang thread-safe dan sangat hemat RAM server.
(MatchHistoryServiceImpl.java)



GLOBAL LEADERBOARD

MMR	WINS	GOALS	SAVES	DEMOS
RANK	USERNAME	SCORE		
1	DREAMER7948	875		
2	PROPLAYER0533	830		
3	SPEEDY8834	705		
4	NIGHTOWL2787	700		
5	DREAMER3402	700		
6	THUNDER2776	690		

BACK

READY FOR
KICKOFF?

